Quantum resistance and Zero knowledge proofs

A learning group for ZK and SNARK application development

Daniel Szego

In: https://www.linkedin.com/in/daniel-szego/



Respectful collaboration starts here



www.lfdecentralizedtrust.org/code-of-conduct

Agenda

- *Logistics*
- *Zero knowledge proofs - summary*
- *High level flow of zero knowledge programming*
- *Zoo of SNARK*
- *Quantum Computers*
- *Quantum Algorithms*
- *SNARK vs STARK*
- *Lattice based zero knowledge proofs*
- *PQC migration of zero knowledge systems*
- *Links, Resources, Literature*
- *Q&A*

Logistics

Every month or two months

Similar structure, similar logistics + ***guest lectures***

Theory:

- Look tables,
- Zk variations: STARKs, FRI
- recursive SNARKs



Programming / platforms:

- ZoKrates, Leo
- Zinc / ZKSync

Applications:

- ZK bridges
- ZK Virtual Machines
- ZK Identity / verifiable credential - presentations
- Voting

Discord channel:

https://discord.com/channels/905194001349627914/1329201532628898036

Meetup.com:

https://www.meetup.com/lfdt-hungary/events/305634614/

Repo with all the contents:

https://github.com/LF-Decentralized-Trust-labs/zk-learning-group

Zero knowledge proofs - summary

Verifiable computation with cryptographic proof

Scarce resource:
- limited verification time: no re-execution
- storage limits: smaller proof
- limited information

Computation: arithmetic circuit : *C(x, w) -> F*
- *x* public input
- *w* private input, witness
- high level computation
- arithmetic circuit
- polynomials

Prover algorithm: *P (pp, x, w) -> proof*

Verifier algorithm: *V (vp, x, proof) -> accept / reject*



Interactive / non-interactive proofs

High level flow of zero knowledge programming

Computation:

- “High” level programs
- Input-output, program instructions

Arithmetization:

- Arithmetic constraint system
- Equations on finite fields
- Constraint system solution: valid computation

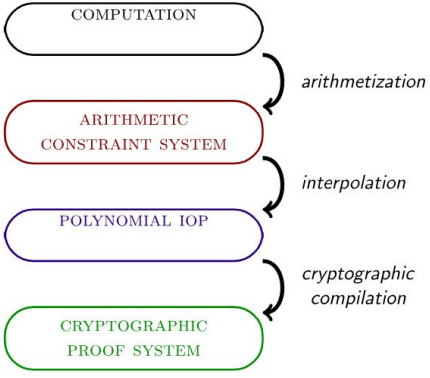
Polynoms

- Polynomial IOP (interactive oracle proof)
- Information theoretic - unconditional security

Proof system:

- Polynomial commitments

- Cryptographic object - security based on computation



Zoo of SNARK

Construction

Several polynomial IOP

Several polynomial commitments

Mixing up components:
- time (verification, proof)
- proof size
- setup assumptions
- post quantum

Polynomial IOP

Interactive proof:
- vSQL, Libra

Multi-prover interactive proofs:
- Spartan

constant -round polynomial IOPs:
- Marlin, Plonk

.

Polynomial commitment

Pairing *:
- KZG10

Discrete logarithm *:
- Bulletproof*

Groups of unknown order *:
- DARK

Hashing:
- FRI

Quantum computers

Wave - particle duality at subatomic level

High level computational abstraction of complex quantum mechanics

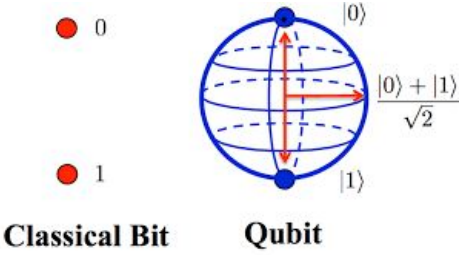
Bit: value is either one or zero

Qubit (superposition): both zero and one and all possible values between zero and one.

Measurement: quantum state collapses into zero or one.

N qubit can represent 2^N state parallely

Quantum advantage: extremely parallel computation but only certain problems



Qubit: https://en.wikipedia.org/wiki/Qubit

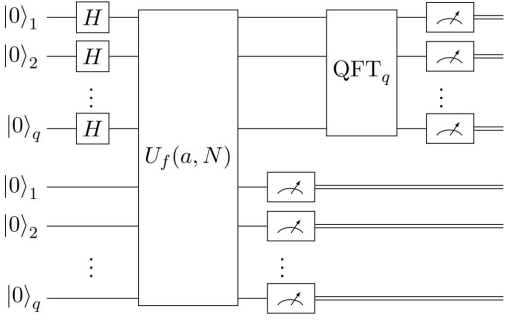
Quantum Algorithms

Solves only certain problems faster than classical computers.

Shor's algorithm (Peter Shor):
- factoring large numbers extremely fast
- polynomial time

Grover's algorithm:
- Quantum search algorithm,
- search an unsorted list of size n for a desired value
- interference amplification of the searched value
- square root performance increase

BB84 Protocol:
- Quantum key distribution protocol
- Quantum mechanical protection against eavesdropping



SNARK vs STARK*





SNARK

- **Succinct:**

  Small in size and fast to verify (from ~300 bytes, under under 10 milliseconds verification time)

  However, proof generation might be resource intensive

- **Non-interactive:**

  No interaction between the prover and the verifier, the prover publishes the proof and the verifier either accepts or rejects

- **Argument of Knowledge:**

  The prover knows a fact to “prove”

Trusted setup: might be a workaround generating in a multiparty way with power of tau

STARK

- **Scalable:**

  Generating proof is fast (quasilinear, (O(nlog {2}n)))

  Verification time is polylogarithmic: 20–30+ milliseconds

  Different computational model (virtual machine)

- **Transparent:**

  No trusted setup

- **Argument of knowledge:**

  The prover knows a fact to “prove”

It might be interactive or non-interactive

Bigger proof size: ~50 - 200 kbytes

Quantum security ! Based on the security of the hash function

Lattice based zero knowledge proofs

Experimental, strongly under research and development

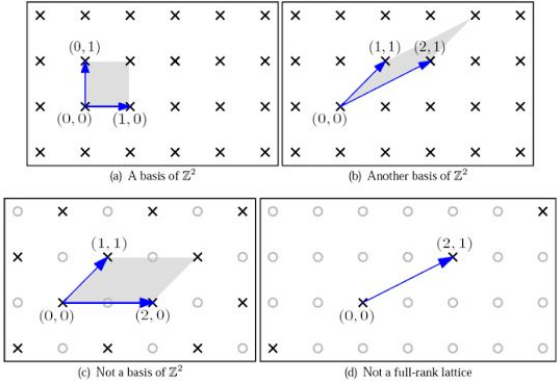
Hard lattice problems:

- Learning with errors (LWE)
- Short integer solution (SIS)

Larger proof size, but fast to verify

Existing experimental frameworks and libraries:

- LaZer
- Lazarus
- LatticeFold / LatticeFold+
- Lantern
- LBZKP



(a) A basis of $\mathbb{Z}^2$ (b) Another basis of $\mathbb{Z}^2$

(c) Not a basis of $\mathbb{Z}^2$ (d) Not a full-rank lattice

PQC migration of zero knowledge systems

E.g. Rollups

Cryptographic inventory of the systems

CBOM - Cryptographic bill of mate

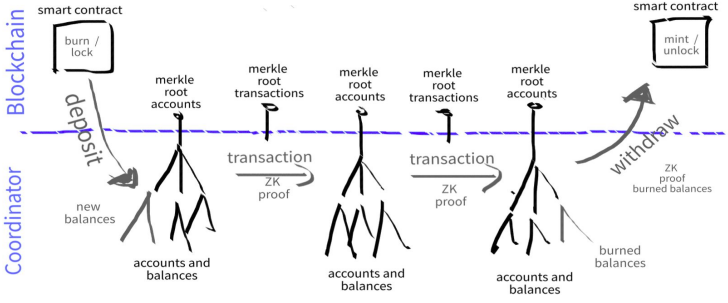
PQC or dual, Hybrid migration

Signatures

Zero knowledge proofs

Upgrade smart contracts

Other elements, like hashes, encryption, TLS ?



Links, Resources, Literature

Zero-Knowledge Proofs in Blockchain Becoming Quantum Secure
https://www.quantumcanary.org/insights/zero-knowledge-proofs-in-blockchain-becoming-quantum-secure

Zk & Quantum Computing: What You Need to Know
https://www.quantumcanary.org/insights/zk-quantum-computing-what-you-need-to-know

Post-Quantum Aztec
https://aztec.network/blog/post-quantum-aztec

How to Build a Post-Quantum Cryptography Migration Roadmap: A 12-Step Playboo
https://quantumgate.ae/post-quantum-cryptography-migration-roadmap

Enabling a Smooth Migration towards Post-Quantum Security for Ethereum
https://pqcee.github.io/Enabling_a_Smooth_Migration_towards_Post_Quantum_Security_for_Ethereum.pdf

License: Apache 2.0

Happy Hunting for the SNARK :)

Q & A

Daniel Szego

In: https://www.linkedin.com/in/daniel-szego/

